SENG 34213 – System Development Project

OWASP Compliance Evidence

OneCare Systems

Frontend, Backend and Chatbot

Version: v 0.1

Date: 25/10/2026

Contents

1. Introduction 1

1.1 Scope 1

1.2 Status key 2

1.3 Evidence rules 2

2. Overall Compliance Summary 3

3. Backend (Spring Boot) – OWASP Top 10 (2021) 4

4. Frontend (React) - OWASP Top 10 (2021) 8

5. Chatbot – OWASP Top 10 for LLM Applications (2025) 13

6. Security Test Evidence 18

6.2 CI/CD Security and Quality Checks 21

7. Known Gaps and Remediation Plan 24

1. Introduction

This document presents the OWASP security compliance assessment of the OneCare Clinic Integrated Pharmacy Management System, following the SENG 34213 System Development Project guidelines (Section 8.1). It covers three main components: the Spring Boot backend, React frontend, and Django-based AI chatbot.

The assessment reviews the OWASP Top 10 (2021) security risks for the backend and frontend and the OWASP Top 10 for LLM Applications (2025) for the chatbot. It describes the implemented security controls, supporting test evidence, identified vulnerabilities, and planned improvements.

Since OneCare handles sensitive patient and medical information, special attention is given to authentication, role-based access control, data protection, secure API communication, and AI chatbot safety. The purpose of this document is to evaluate the system's security measures, identify remaining gaps, and support improvements before production deployment.

1.1 Scope

This OWASP compliance assessment covers the three main components of the OneCare Clinic Integrated Pharmacy Management System: the backend, frontend, and AI chatbot. Each component is evaluated against the relevant OWASP security standards.

Component	Technology	Standard applied
Backend	Java, Spring Boot REST API, Spring Security, Spring Data JPA, MySQL, Redis	OWASP Top 10 (2021)
Frontend	React, Redux Toolkit, React Router, Vite, Tailwind CSS, Axios	OWASP Top 10 (2021), client-side security
Chatbot	Python, Django REST Framework, Retrieval-Augmented Generation (RAG), FAISS, Sentence Transformers (LaBSE), external LLM API, MySQL	OWASP Top 10 for LLM Applications (2025)

Table 1: Technology selection Component vise

The assessment focuses on authentication, authorization, input validation, sensitive data protection, secure API communication, dependency security, and AI chatbot safety. It also reviews available security test evidence and identifies areas requiring further improvement.

1.2 Status key

The following status categories are used to indicate the implementation and verification level of each OWASP security control.

Status	Meaning
Implemented	The security control has been implemented and verified through testing or code review, with supporting evidence available.
Partial	The security control is implemented but requires further improvements or verification. Remaining gaps are documented in Section 7.
Planned	The security control has not yet been implemented and is included in the planned development or security improvement activities.
N/A	Risk does not apply to this component; justification provided.

Table 2: status key explanation

These statuses help identify completed security controls, remaining security gaps, and improvements required before production deployment.

1.3 Evidence rules

The following rules are followed to ensure that the OWASP security assessment is accurate, reliable, and supported by evidence.

- **Supporting Evidence:** Each OWASP security control must include relevant evidence, such as GitHub issues, pull requests, source code files, test files, CI/CD pipeline results, configuration files, or screenshots.
- **Test Evidence:** Security testing results should be linked to the relevant test case IDs in the Test Case Register (documents/testing/test-register.pdf).
- **Sensitive Data Protection:** Real passwords, API keys, authentication tokens, patient information, and other sensitive data must not be included in screenshots, logs, or reports. Sensitive information must be removed or masked before documentation.
- **Evidence Verification:** A security control should be marked as **Implemented** only when its implementation and supporting evidence have been verified. Controls requiring further testing or improvement should be marked as **Partial**.

- **Security Gap Tracking:** Any incomplete or unverified security controls must be documented in **Section 7 – Known Gaps and Remediation Plan**, together with the required improvements.

2. Overall Compliance Summary

This section summarizes the OWASP security compliance status of the three main components of the OneCare Clinic Integrated Pharmacy Management System: Backend, Frontend, and AI Chatbot.

The assessment covers the OWASP Top 10 (2021) for the Backend and Frontend and the OWASP Top 10 for LLM Applications (2025) for the Chatbot.

The following counts are calculated from the security assessment tables in Sections 3, 4, and 5.

Component	Risks	Implemented	Partial	Planned	N/A
Backend (Spring Boot)	10	4	6	0	0
Frontend (React)	10	3	7	0	0
Chatbot (LLM)	10	2	8	0	0
Total	30	9	21	0	0

Table 3: Overall OWASP Compliance Summary

The assessment identifies 30 security risk categories across the three components. Based on the current assessment, 9 categories are marked as Implemented and 21 as Partial.

The system includes security controls such as authentication, role-based access control, input validation, secure API communication, and AI chatbot safety mechanisms. However, some controls require additional testing, security improvements, and verification.

The remaining security gaps and planned improvements are documented in Section 7 – Known Gaps and Remediation Plan.

3. Backend (Spring Boot) – OWASP Top 10 (2021)

The Spring Boot backend is responsible for enforcing the main security controls of the OneCare Clinic Integrated Pharmacy Management System. It manages authentication, authorization, input validation, database operations, and secure communication between system components.

The following table summarizes the security measures implemented to address the OWASP Top 10 (2021) risks, along with supporting evidence and the current compliance status.

Risk	Mitigation implemented	Evidence	Status
A01 Broken Access Control	Default-deny with anyRequest().authenticated() and role rules per route group. @PreAuthorize permission checks on every controller method. Doctors can only read or edit their own appointments and clinical notes. Only the issuing doctor can update a prescription. Prescribing doctor is taken from the token, not the request body. Clients cannot send status, totals or prices. CORS origin allow-list.	config/SecurityConfig.java security/RolePermission.java RbacAccessControlTest PrescriptionControllerIntegrationTest (create_asAdmin_returns403) AppointmentApiIntegrationTest (clinicalNotes_asOtherDoctor_returns403) CorsConfigurationTest	Implemented
A02 Cryptographic Failures	Passwords hashed with Argon2id (64 MB, t=2, p=1). Reset tokens from SecureRandom, stored only as SHA-256 hashes. Redis blacklist keys use a token hash. JWT secret and DB credentials come from environment variables.	SecurityConfig.passwordEncoder() JwtAuthenticationIntegrationTest (passwordEncoderUsesConfiguredArgon2idParameters) PasswordResetServiceImplTest TokenBlacklistServiceTest	Partial

A03 Injection	JPA derived queries, JPQL with @Param, Specifications and a parameterised native query only. @Valid Bean Validation on request DTOs. Enums for typed input (role, payment method, status).	repository/*Repository.java dto/request/* PasswordResetApiIntegrationTest (validationFailureShouldReturn400WithFieldErrors) UserControllerIntegrationTest (invalidRole_returns400) InvoiceApiIntegrationTest (invalidPaymentAmountsReturn400AndChangeNothing)	Implemented
A04 Insecure Design	Status and totals are derived server-side, and invoice prices are snapshotted. Optimistic locking (@Version) on payments and a pessimistic lock on doctor slots. DB constraints: unique doctor slot, one invoice per appointment, CHECKs. Soft deletes, cash-only payments, overpayment rejected. Forgot-password gives the same response for any email	InvoicePaymentConcurrencyIntegrationTest AppointmentConcurrencyIntegrationTest schema.sql PrescriptionControllerIntegrationTest (create_ignoresClientSuppliedStatusAndDoctorId) PasswordResetApiIntegrationTest (forgotPasswordShouldReturnIdenticalResponse...)	Implemented
A05 Security Misconfiguration	Separate dev, staging and prod profiles. Prod: ddl-auto=validate, sql.init.mode=never, SQL logging off, Swagger off. Generic “Internal Server Error” with	application-prod.properties GlobalExceptionHandler.java .env-example .github/workflows/ci.yml	Partial

	no stack traces. No Actuator exposed. Secrets via environment variables, .env-example and GitHub Secrets. Stateless sessions.		
A06 Vulnerable and Outdated Components	Spring Boot 3.5.14, Java 21 LTS and jjwt 0.12.6. Maven version pinned through the wrapper.	pom.xml .mvn/wrapper/maven-wrapper.properties CI dependency scan: none yet	Partial
A07 Identification and Authentication Failures	Lockout after 5 failed attempts for 15 min, with an atomic counter and reset on success. Reset tokens are single-use, expire in 30 min, and old ones are invalidated. JWT signature, expiry and type (access vs refresh) validation. Logout blacklists the access token in Redis with a TTL. Generic login errors and rate limiting on /api/auth/*.	UserServiceImpl.recordFailedAttempt AuthServiceImplTest UserServiceImplTest JwtAuthenticationIntegrationTest (logout and blacklist tests) RateLimitFilterIntegrationTest ApiRateLimitFilterTest	Partial
A08 Software and Data Integrity Failures	Signed and verified JWTs. CI runs build and tests on PRs to main and develop. Transactions, optimistic locking and immutable paid invoices. No untrusted Java deserialization.	JwtService.java .github/workflows/ci.yml entity/Invoice.java (@Version) InvoiceApiIntegrationTest (lineEdit_onPaidInvoiceReturns400...)	Partial
A09 Security Logging and	SLF4J logs for logins, failures, lockouts, role changes, user and patient changes, prescriptions, invoices, payments and password	AuthServiceImpl.java UserServiceImpl.java ExternalDispensingServiceImpl.recordAudit	Partial

Monitoring Failures	resets, each with performedBy. No passwords or tokens logged. External dispensing writes persisted AuditLog rows.	InvoiceServiceImplTest (emitsExactlyOneAuditRecordForGenerateEditAndPay)	
A10 Server-Side Request Forgery (SSRF)	No user-supplied URL is fetched anywhere. chatbot.url is fixed server config with connect and read timeouts. Reset link uses the configured frontend base URL, not the Host header.	application-*.properties (chatbot.url, timeouts) EmailServiceImpl.java EmailServiceImplTest	Implemented

Table 4: Backend OWASP Top 10 Security Assessment

Backend Security Assessment Summary

Status	Number of Risks
Implemented	4
Partial	6
Planned	0
N/A	0
Total	10

Table 5: Backend Security Assessment Summary

The backend includes important security controls such as role-based access control, secure password hashing, JWT authentication, input validation, database transaction protection, and audit logging.

Based on the documented assessment, four OWASP risk categories are marked as Implemented, while six are marked as Partial. The remaining work mainly involves production security verification, dependency vulnerability scanning, secure communication, authentication testing, and security monitoring.

These areas should be addressed before production deployment. The remaining gaps and recommended improvements are documented in Section 7 – Known Gaps and Remediation Plan.

4. Frontend (React) - OWASP Top 10 (2021)

The React frontend provides security features to protect user sessions, validate inputs, and control access to different pages. However, frontend security controls can be bypassed, so the Spring Boot backend must independently enforce authentication, authorization, and input validation.

The following table summarizes the frontend security controls, supporting evidence, and current compliance status.

Risk	Mitigation	implemented Evidence	Status
A01 Broken Access Control	RequireAuth guard: unauthenticated users go to /login (return path kept), wrong role goes to their own dashboard, unknown role goes to /login. Central ROUTE_ACCESS table, with every route group (admin, doctor, pharmacist, super-admin, shared queue) wrapped in the guard. Sidebar menu is role-driven (NAV_ITEMS), and a test checks every menu entry is allowed for its role. Guards are documented as a UX layer only; the backend authorises every request. 403 responses are shown to the user without logging them out.	src/routes/RequireAuth.jsx src/constants/routeAccess.js src/constants/navigation.js test/routes/RequireAuth.test.jsx test/routes/appRoutes.integration.test.jsx test/routes/navigation.test.js test/components/Sidebar.test.jsx test/services/apiClient.test.js (AC5: 403/429/500)	Implemented
A02 Cryptographic Failures	Access token attached in one place only (request interceptor) and never logged.	src/services/apiClient.js src/services/config.js .env.example	Partial

	Passwords and reset tokens sent only in POST bodies over the configured API URL. Reset token is removed from the browser URL right after it is read. .env.example holds only the public VITE_API_BASE_URL; no secrets in the client.	test/services/authService.test.js (“never logs the password”) test/pages/public/ResetPasswordPage.test.jsx (token removed from URL) src/app/store.js (persist config, see gap below)	
A03 Injection (incl. XSS)	React’s default output escaping; no dangerouslySetInnerHTML in the code you shared. Clinical notes and prescription text render as plain text (whitespace-pre-wrap). Form validation: required fields, email format, password 8–64 characters, confirm-password match, notes maxLength 2000, numeric minimums on prescriptions. Query strings built with Axios params (encoded). Open-redirect guard: login target must start with / and not //. Backend re-validates everything.	src/components/Input.jsx src/pages/public/LoginPage.jsx src/pages/public/ResetPasswordPage.jsx src/pages/doctor/ConsultationPage.jsx test/components/formValidation.integration.test.jsx test/components/Input.test.jsx test/pages/doctor/ConsultationPage.test.jsx (2000-char cap) test/routes/appRoutes.integration.test.jsx (return-to-route)	Implemented
A04 Insecure Design	Idle timeout of 30 min with a warning 2 min before, then logout and local clear. Double-submit guards (disabled and loading buttons, completingRef on Complete Consultation). Notes and prescription forms	src/features/auth/SessionManager.jsx src/features/auth/sessionActivity.js src/pages/doctor/ConsultationPage.jsx src/pages/doctor/KnowledgeBasePage.jsx	Partial

	locked on completed or cancelled appointments. Confirmation dialog before removing a Knowledge Base reference. Polling pauses while the tab is hidden. Client validation is convenience only; the server stays authoritative.	src/hooks/useQueuePolling.js test/features/auth/sessionManager.test.jsx test/pages/doctor/ConsultationPage.test.jsx (duplicate completion click) test/hooks/useQueuePolling.test.jsx	
A05 Security Misconfiguration	Only VITE_API_BASE_URL is exposed to the client; .dockerignore excludes .env*. Docker image runs as the non-root node user and installs with npm ci. Debug logging only when import.meta.env.DEV. External links use rel="noopener noreferrer". Users see generic error messages, never stack traces.	Dockerfile .dockerignore .env.example src/services/apiClient.js src/pages/doctor/KnowledgeBasePage.jsx	Partial
A06 Vulnerable Components	package-lock.json plus npm ci in both CI and Docker. Current major versions of React, Vite, Axios and Redux Toolkit. CI runs contrast check, lint, tests with coverage and build; Husky and lint-staged are installed.	package.json .github/workflows/ci.yml npm audit / Dependabot: none yet	Partial
A07 Auth Failures	Generic “Invalid username or password.” on 401, and the password field is cleared after any failed attempt. 429 (rate limit) and 423 (lockout)	src/pages/public/LoginPage.jsx src/pages/public/ForgotPasswordPage.jsx src/pages/public/ResetPass	Partial

	have clear messages. Correct autoComplete attributes and an accessible show/hide-password toggle. Forgot-password shows the same neutral confirmation either way. Invalid or used reset links get a dedicated screen. Single-flight token refresh; a failed refresh ends the session once with no retry loop. Idle logout and a “session expired” banner. Logout calls the backend so the access token is blacklisted.	wordPage.jsx src/services/apiClient.js test/pages/public/LoginPage.test.jsx test/pages/public/ForgotPasswordPage.test.jsx test/services/apiClient.integration.test.jsx (AC2/AC3) test/features/auth/sessionManager.test.jsx	
A08 Integrity Failures	npm ci enforces the lock file, and CI runs on PRs to main and develop. No third-party scripts (only a Google Fonts stylesheet). API responses pass through adapter functions that check their shape before the UI uses them. Temporary PDF blob URLs are revoked after use.	.github/workflows/ci.yml src/services/queueService.js src/services/appointmentService.js src/services/patientService.js test/services/queueService.test.js test/services/appointmentService.test.js	Partial
A09 Logging Failures	Tokens, passwords and request bodies are never logged (documented and tested). Errors are normalised to the standard {success, message, data} envelope. Session-expiry event handled in	src/services/apiClient.js test/services/authService.test.js test/services/apiClient.test.js (session-expired event fires once)	Partial

	one place. Debug output only in development.		
A10 SSRF	All HTTP goes through one Axios client with a fixed base URL, and components contain no hostnames. No user-supplied URL is fetched; PDF viewer sources are only blob: URLs or bundled /pdfs/... paths. The only navigation to a dynamic target is the validated login redirect.	src/services/apiClient.js src/services/config.js src/pages/public/LoginPage.jsx src/pages/doctor/KnowledgeBasePage.jsx	Implemented

Table 6: Frontend OWASP Top 10 Security Assessment

Frontend Security Assessment Summary

Status	Number of Risks
Implemented	4
Partial	6
Planned	0
N/A	0
Total	10

Table 7: Security Assessment Summary Table

The frontend includes important security measures such as protected routes, role-based navigation, input validation, session management, and secure API request handling.

Based on the documented assessment, three OWASP risk categories are marked as Implemented and seven as Partial. Further verification is required for token storage, production configuration, dependency security, authentication handling, and monitoring.

The remaining security gaps and recommended improvements are documented in Section 7 – Known Gaps and Remediation Plan.

5. Chatbot – OWASP Top 10 for LLM Applications (2025)

The OneCare AI chatbot uses Retrieval-Augmented Generation (RAG) to provide general clinic information in English and Tamil. It uses Django REST Framework, FAISS vector search, Sentence Transformers, and an external LLM API.

The chatbot follows the OWASP Top 10 for LLM Applications (2025) to reduce risks related to prompt injection, sensitive information disclosure, unsafe responses, and misuse of AI-generated information. Standard API security controls, including authentication, input validation, and rate limiting, are also applied.

The following table summarizes the implemented security measures, supporting evidence, and current compliance status.

Risk	Mitigation implemented	Evidence (replace with links)	Status
LLM01 Prompt Injection	User questions and retrieved information are treated as untrusted. The system removes harmful instructions and keeps information separate from system instructions. The AI is told not to follow instructions found in retrieved content. Medical safety checks and response filtering provide extra protection.	chat_pipeline/prompts.py, resources/system_prompt.txt; tests/test_chat08_prompt.py	Partial
LLM02 Sensitive Information Disclosure	Internal chatbot APIs are protected using a shared security token, which is checked securely. Before saving chat logs, personal information such as email addresses, ID numbers, and long number sequences is hidden. LLM API keys are stored securely in	onecare_chat/service_auth.py, chat_pipeline/chat_log.py; tests/test_chat10_logging.py	Partial

	environment variables instead of directly in the code..		
LLM03 Supply Chain	Python package versions are fixed to avoid unexpected changes. The CI pipeline checks package versions, runs tests to find errors, and uses pip-audit to check for known security vulnerabilities in the installed packages..	requirements.txt, .github/workflows/ci.yml; tests/test_check_pins.py	Partial
LLM04 Data and Model Poisoning	Only users with DOCTOR or SUPER_ADMIN roles can manage the chatbot knowledge base. Uploaded documents and website links are checked for safety and validity. The chatbot retrieves information only from the latest active versions of documents stored in the knowledge base.	Backend KnowledgeBaseController.java; knowledgebase/validators.py, url_safety.py, retrieval.py	Partial
LLM05 Improper Output Handling	Generated answers pass through a filter that blocks diagnosis, dosage, prompt leakage and configured off-scope content. Filter errors fail closed and return a fallback. The chat widget renders server text using textContent.	chat_pipeline/post_filter.py; tests/test_chat06_post_filter.py; frontend onecare-chat-widget.js	Implemented

LLM06 Excessive Agency	The LLM chatbot is used only to generate text responses. The application code controls how information is retrieved from the knowledge base and clinic system. The chatbot cannot directly prescribe medicine, book appointments, or change patient medical records.	chat_pipeline/llm.py, generation.py, service.py	Implemented
LLM07 System Prompt Leakage	The chatbot is instructed to keep its system instructions secret. It checks generated responses to prevent these instructions from being revealed. If any sensitive system instructions are detected, the chatbot replaces the response with a safe fallback message.	resources/system_prompt.txt, chat_pipeline/post_filter.py; tests/test_chat06_post_filter.py	**P**artial
LLM08 Vector and Embedding Weaknesses	The chatbot retrieves information only from active and indexed documents using their current versions. It uses the same embedding model to understand user questions and document content. The number of search results is limited to avoid unnecessary information and improve performance.	knowledgebase/retrieval.py, embedding.py; chat_pipeline/search.py; tests/test_chat04_search_db.py	Partial
LLM09	The chatbot generates answers using retrieved	chat_pipeline/service.py, confidence.py, guardrail.py;	Partial

Misinformation	information and checks its confidence level. If it cannot find reliable information or the AI fails to generate an answer, it provides a safe fallback message. The application code controls the information sources. The chatbot refuses requests for medical diagnosis, prescriptions, and medicine dosages to ensure patient safety..	tests/test_chat05_fallback.py, test_chat02_guardrail.py	
LLM10 Unbounded Consumption	Controls include a default 500-character query limit, public API rate limiting of 20 requests per minute per IP, bounded context and retrieval sizes, LLM output-token limits, timeouts and limited retries.	chat_pipeline/views.py, prompts.py, llm.py; backend ChatApiConfig.java, ChatApiProperties.java	Partial

Table 8: Chatbot OWASP Top 10 for LLM Applications Security Assessment

Chabot Security Assessment Summary

Status	Number of Risks
Implemented	2
Partial	8
Planned	0
N/A	0
Total	10

Table 9: Chatbot Security Assessment Summary Table

The OneCare chatbot includes important security controls such as service-token authentication, knowledge-base validation, medical safety guardrails, response filtering, and API rate limiting.

Based on the documented assessment, two OWASP risk categories are marked as Implemented, while eight are marked as Partial.

The remaining improvements mainly involve prompt injection testing, sensitive information protection, dependency scanning, knowledge-base trust verification, chatbot response accuracy, and resource-consumption testing.

These security gaps and recommended improvements are documented in Section 7 – Known Gaps and Remediation Plan.

6. Security Test Evidence

This section describes the security tests and automated CI/CD checks used to assess the OneCare Clinic Integrated Pharmacy Management System.

Security testing covers the Spring Boot backend, React frontend, and Django AI chatbot. The tests focus on authentication, authorization, input validation, data protection, dependency security, and AI chatbot safety.

Each test case should be linked to its source code, automated test file, and CI/CD execution results. The test case IDs should also be recorded in the Test Case Register (documents/testing/test-register.pdf).

Component	Risk	Scenario	Expected result	Test file
Backend	A01-Broken Access Control	A user without the required role attempts to access a restricted API.	HTTP 403 Forbidden is returned.	`RbacAccessControlTest`
Backend	A01 – Broken Access Control	A doctor attempts to access another doctor's clinical notes.	Access is denied with HTTP 403.	`AppointmentApiIntegrationTest`
Backend	A03 – Injection	Submit invalid role values or incorrectly formatted request data	Invalid input is rejected with HTTP 400.	`UserControllerIntegrationTest`
Backend	A04 – Insecure Design	Two users attempt to book the same appointment slot simultaneously	Duplicate bookings are prevented.	`AppointmentConcurrencyIntegrationTest`

Backend	A07 – Authentication Failures	Submit repeated incorrect login credentials.	Account lockout is applied after the configured limit.	`UserServiceImplTest`
Backend	A07 – Authentication Failures	Attempt to reuse a JWT after logout.	The blacklisted token is rejected.	`JwtAuthenticationIntegrationTest`
Frontend	A01 – Broken Access Control	An unauthenticated user opens a protected page.	The user is redirected to the login page.	`RequireAuth.test.jsx`
Frontend	A01 – Broken Access Control	A user attempts to open a page restricted to another role.	Unauthorized navigation is prevented.	`appRoutes.integration.test.jsx`
Frontend	A03 – Injection	Submit invalid or excessive form input.	Validation messages are displayed, and invalid input is rejected.	`formValidation.integration.test.jsx`
Frontend	A07 – Authentication Failures	Token refresh fails during an API request.	The session ends without repeated refresh attempts.	`apiClient.integration.test.jsx`
. Chatbot	LLM01 – Prompt Injection	Submit malicious instructions through a user question or retrieved document.	Untrusted instructions do not override the chatbot's system rules.	`tests/test_chat08_prompt.py`

Chatbot	LLM02 – Sensitive Information Disclosure	Log a message containing email addresses and identification numbers.	Supported sensitive information patterns are masked	`tests/test_chat10_logging.py`
Chatbot	LLM05 – Improper Output Handling	Generate a response containing restricted medical instructions.	Unsafe output is blocked and replaced with a safe response.	`tests/test_chat06_post_filter.py`
Chatbot	LLM07 – System Prompt Leakage	Ask the chatbot to reveal internal system instructions.	Protected instructions are not disclosed.	`tests/test_chat08_prompt.py`
Chatbot	LLM08 – Vector and Embedding Weaknesses	Retrieve information from inactive knowledge-base documents.	Inactive document versions are excluded.	`tests/test_chat04_search_db.py`
Chatbot	LLM09 – Misinformation	Ask a question without relevant knowledge-base information.	A safe fallback response is returned.	`tests/test_chat05_fallback.py`
Chatbot	LLM09 – Misinf	Request medical diagnosis, prescription	The chatbot refuses the unsafe medical request.	`tests/test_chat02_guardrail.py`

	ormatio n	, or dosage advice.		

Table 10: Security Test Cases and Expected Results

6.2 CI/CD Security and Quality Checks

The OneCare project uses GitHub Actions to automate security and quality checks for its three components. The workflows run on pushes and pull requests targeting the develop and main branches.

Component	CI/CD Check	Implementation	Expected Result
Backend	Code quality	Maven Checkstyle	Code passes the configured style rules.
Backend	Static analysis	SpotBugs	No findings that cause the configured check to fail.
Backend	Secret scanning	Gitleaks	No detected secrets that violate the scan rules.
Backend	Dependency review	GitHub Dependency Review on pull requests	No dependency changes that violate the high-severity threshold.
Backend	Vulnerability scanning	OWASP Dependency-Check	When enabled, the scan fails for vulnerabilities with CVSS 7 or higher.
Backend	Unit and integration tests	Maven clean test, MySQL 8.4	All required tests pass.
Backend	Code coverage	JaCoCo	Overall coverage is at least 70%.
Backend	Docker build	Docker Buildx	Backend Docker image builds successfully.
Frontend	Translation validation	npm run check:i18n	Patient translation keys pass validation.
Frontend	Accessibility check	npm run check:contrast	Configured colour-contrast checks pass.
Frontend	Code quality	ESLint	No lint errors.

Frontend	Automated tests	Vitest coverage	Tests pass and configured coverage thresholds are met.
Frontend	Dependency scanning	npm audit --omit=dev --audit-level=high	No reported high or critical vulnerabilities in audited production dependencies.
Frontend	Dockerfile validation	Hadolint	Dockerfile passes linting.
Frontend	Secret scanning	Gitleaks on the built frontend bundle	No secrets detected in the shipped bundle.
Frontend	Docker vulnerability scanning	Trivy	No fixable HIGH or CRITICAL vulnerabilities are detected.
Frontend	Container publishing	GitHub Container Registry (GHCR)	Only successful branch-push workflows publish SHA-tagged images.
Chatbot	Dependency version validation	scripts/check_pins.py	Required Python dependency pins pass validation.
Chatbot	Code quality	Ruff	No lint errors.
Chatbot	Automated tests	Pytest	Selected test groups pass.
Chatbot	Dependency vulnerability scanning	pip-audit with OSV	The dependency audit completes successfully.
Chatbot	Embedding tests	Pytest embedding marker	Embedding tests pass.
Chatbot	Database integration	MySQL 8.0, Django migrations	Database setup and migration checks pass.
Chatbot	Code coverage	Pytest with coverage reporting	Coverage results are generated for the knowledge-base module.
Chatbot	Docker build	Docker	Chatbot Docker image builds successfully.

Table 11: CI/CD Security and Quality Checks

6.3 CI/CD Verification and Evidence

The CI/CD pipelines provide automated checks during development and pull request reviews.

- The Backend pipeline includes Checkstyle, SpotBugs, Gitleaks, dependency review, OWASP Dependency-Check, automated testing, JaCoCo coverage verification, and Docker image building.
- The Frontend pipeline includes translation checks, accessibility checks, linting, automated tests, dependency auditing, Dockerfile linting, secret scanning, and container vulnerability scanning. Docker images are published using commit SHA tags after the required checks succeed.
- The Chatbot pipeline includes dependency pin validation, Ruff linting, Pytest, vulnerability auditing, embedding tests, database migration verification, coverage reporting, and Docker image building.

CI/CD evidence should include the relevant GitHub Actions workflow run, execution date, commit ID, and test or scan report. Failed or skipped checks must be identified and should not be reported as passed.

Any unresolved security findings must be documented in Section 7 – Known Gaps and Remediation Plan.

7. Known Gaps and Remediation Plan

This section identifies the remaining security gaps in the OneCare Clinic Integrated Pharmacy Management System and describes the improvements required to address them.

The assessment covers the Spring Boot backend, React frontend, and Django AI chatbot. Each security risk marked as Partial or Planned in Sections 3, 4, and 5 is included in the remediation plan.

The target dates below are proposed milestones and should be confirmed by the development team before final submission.

Gap ID	Description	Severity	Component	Target
G-BE-01	A02 – Cryptographic Failures: Verify HTTPS/TLS configuration, secure transmission of sensitive information, and production secret management.	High	Backend	Before production release
G-BE-02	A05 – Security Misconfiguration: Verify production configuration, disabled debugging features, environment variables, and security settings before deployment.	High	Backend	Before production release
G-BE-03	A06 – Vulnerable Components: Confirm that OWASP Dependency-Check runs successfully and review or resolve reported high-severity vulnerabilities.	High	Backend	Before production release
G-BE-04	A07 – Authentication Failures: Complete additional testing of account lockout, JWT expiration, token refresh, and authentication rate limiting.	High	Backend	Before production release

G-BE-05	A08 – Integrity Failures: Verify dependency integrity, CI/CD security checks, and protection of important application data.	Medium	Backend	Before next release
G-BE-06	A09 – Logging and Monitoring: Improve security event monitoring, verify sensitive-data redaction, and establish log review and retention procedures.	Medium	Backend	Before next release
G-FE-01	A02 – Cryptographic Failures: Review authentication token storage, confirm HTTPS usage, and ensure sensitive information is not exposed in the browser.	High	Frontend	Before production release
G-FE-02	A04 – Insecure Design: Complete additional testing of session timeout, duplicate submissions, and restricted workflows.	Medium	Frontend	Before next release
G-FE-03	A05 – Security Misconfiguration: Verify production configuration, browser security headers, and frontend environment settings.	High	Frontend	Before production release
G-FE-04	A06 – Vulnerable Components: Verify successful npm audit and Trivy scans, and address reported fixable high or critical vulnerabilities.	High	Frontend	Before production release
G-FE-05	A07 – Authentication Failures: Test session expiration, failed token refresh, logout, and authentication error handling.	High	Frontend	Before production release

G-FE-06	A08 – Integrity Failures: Verify dependency integrity, CI/CD checks, and security of the production build.	Medium	Frontend	Before next release
G-FE-07	A09 – Logging and Monitoring: Verify that sensitive information is not logged and improve frontend error monitoring.	Medium	Frontend	Before next release
G-CHAT-01	LLM01 – Prompt Injection: Complete adversarial testing in English and Tamil, including malicious user questions and retrieved documents attempting to override system instructions.	High	Chatbot	Before production release
G-CHAT-02	LLM02 – Sensitive Information Disclosure: Improve personal and medical information redaction, verify information sent to external LLM services, and establish secure log retention procedures.	High	Chatbot	Before production release
G-CHAT-03	LLM03 – Supply Chain: Verify successful pip-audit results, resolve reported dependency vulnerabilities, and confirm approved embedding and LLM model versions.	High	Chatbot	Before production release
G-CHAT-04	LLM04 – Data and Model Poisoning: Improve verification of uploaded knowledge-base content and test protection against malicious or misleading documents.	High	Chatbot	Before production release

G-CHAT-05	LLM07 – System Prompt Leakage: Add further tests to ensure internal system instructions cannot be disclosed through direct or indirect prompt attacks.	High	Chatbot	Before production release
G-CHAT-06	LLM08 – Vector and Embedding Weaknesses: Test retrieval against manipulated content and verify isolation of inactive or outdated knowledge-base documents.	Medium	Chatbot	Before next release
G-CHAT-07	LLM09 – Misinformation: Evaluate chatbot response accuracy in English and Tamil and verify safe fallback behaviour for unsupported medical questions.	High	Chatbot	Before production release
G-CHAT-08	LLM10 – Unbounded Consumption: Perform load and resource-consumption tests to verify query limits, rate limiting, timeouts, and LLM token restrictions.	Medium	Chatbot	Before next release

Table 12: Known Security Gaps and Remediation Plan

RemediationSummary

Component	**High Severity**	**Medium Severity**	**Total Gaps**
Backend	4	2	6
Frontend	4	3	7
Chatbot	6	2	8
Total	14	7	21

Table 13: Remediation Summary Description

The assessment identifies 21 security gaps across the three system components. These correspond to the 21 Partial security categories identified in the Overall Compliance Summary.

High-severity gaps should be prioritized before production deployment, especially those related to authentication, sensitive data protection, dependency vulnerabilities, prompt injection, and chatbot response safety.

Medium-severity gaps should be addressed through the planned security improvement activities and verified through additional testing.

Each gap should be assigned to a responsible team member, tracked through GitHub issues, and closed only after the required security improvements have been implemented and verified.